

Sonrisa - Parkolórendszer

User prompt: Egy parkolóhely-foglalási backend rendszert építünk C# nyelven, .NET 10 környezetben, ASP.NET Core Web API használatával. Az adatbázis MS SQL Server lesz, az adateléréshez pedig Entity Framework Core-t használunk. Az alap API projekt vázát már létrehoztam. Kérlek, senior .NET fejlesztői szemlélettel, az iparági 'best practice'-eket szem előtt tartva írd meg nekem a domain modelleket. Kellene egy ParkingSpace és egy Reservation osztály. Egy parkolóhelynek legyen egy int Id-ja és egy típusa, mondjuk normál vagy mozgássérült. Egy foglaláshoz tartozzon hozzá a parkolóhely Id-ja, egy kezdő időpont, egy záró időpont és a kérelmező neve. Kérlek, készítsd el a hozzájuk tartozó EF Core ApplicationDbContext osztályt is a megfelelő DbSet-ekkel. Mivel egyelőre egy alap API projektből indulunk, röviden vázold fel azt is, hogy ezen a projekten belül milyen mappákba érdemes rendezni ezeket a fájlokat.

Response: Örömmel segítek. Egy senior .NET fejlesztői megközelítésnél az entitások tervezésekor a legfontosabb szempontok közé tartozik a típusbiztonság, az egyértelmű kapcsolatok definiálása, a megfelelő adattípusok használata (például időzónák kezelése), valamint az adatbázis-konfigurációk elkülönítése az üzleti logikától.

Az alábbiakban bemutatom a javasolt architektúrát, a modelleket és az Entity Framework Core kontextust, figyelembe véve a legújabb C# nyelvi elemeket, és **teljes mértékben mellőzve a var kulcsszó használatát**.

Mappaszerkezet (Egyprojektes API esetén)

Még ha egyelőre csak egy alap Web API projektből indulsz is, érdemes a Clean Architecture vagy a "Feature-alapú" mappa-struktúra elveit követni, hogy később könnyen skálázható legyen az alkalmazás.

Egy javasolt, logikailag szeparált felépítés:

- **Domain/**
 - Entities/ (Ide kerülnek a modellek: ParkingSpace.cs, Reservation.cs)
 - Enums/ (ParkingSpaceType.cs)
- **Infrastructure/**
 - Data/ (Ide kerül az ApplicationDbContext.cs és a migrációk)
 - EntityConfigurations/ (Később ide érdemes kiszervezni a Fluent API beállításokat)
- **DTOs/** (Data Transfer Objectek, amiket az API a kliens felé küld/fogad)
- **Controllers/** (Vagy Endpoints/, ha Minimal API-t használsz)
- **Services/** (Az üzleti logika, ami az adatbázis és a controllerek között ül)

Domain Modellek

A parkolóhely típusához érdemes egy enum-ot létrehozni a string helyett, ami típusbiztosabb és az adatbázisban is hatékonyabban (számként vagy konvertálva stringként) tárolható.

```
namespace ParkingSystem.Domain.Enums;
```

```
public enum ParkingSpaceType  
{
```

```

        Standard = 0,
        Accessible = 1 // Mozgássérült
    }

```

A modelleknel használjuk a C# újabb funkcióit. Az Id mezőkre init adatkötést teszünk (hiszen a létrehozás után nem változhatnak), a kötelező szöveges mezőkre pedig a required módosítót. Az időpontokhoz a DateTimeOffset használata javasolt a sima DateTime helyett, mivel a foglalásoknál kritikus lehet a pontos időzóna kezelése.

ParkingSpace osztály

```

using System.Collections.Generic;
using ParkingSystem.Domain.Enums;

namespace ParkingSystem.Domain.Entities;

public class ParkingSpace
{
    public int Id { get; init; }
    public ParkingSpaceType Type { get; set; }

    // Navigációs tulajdonság a foglalások eléréséhez
    // Kezdeti érték adása megakadályozza a NullReferenceException-t
    public ICollection<Reservation> Reservations { get; set; } = new List<Reservation>();
}

```

Reservation osztály

```

using System;

namespace ParkingSystem.Domain.Entities;

public class Reservation
{
    public int Id { get; init; }

    // Idegen kulcs
    public int ParkingSpaceId { get; set; }

    // Navigációs tulajdonság (lehet null, amíg az EF be nem tölti)
    public ParkingSpace? ParkingSpace { get; set; }

    // DateTimeOffset használata a pontos idő/időzóna miatt
    public DateTimeOffset StartTime { get; set; }
    public DateTimeOffset EndTime { get; set; }

    // A 'required' biztosítja, hogy a példányosításkor meg kell adni a nevet
    public required string ApplicantName { get; set; }
}

```

Entity Framework Core Kontextus (AppDbContext)

A modellek közötti kapcsolatokat (például, hogy egy parkolóhelyhez több foglalás tartozhat) adatbázis-szinten a modelBuilder segítségével, a "Fluent API" használatával érdemes konfigurálni. Ez tisztábban tartja a domain modelleket, mintha tele lennének szórva Data Annotation attribútumokkal (pl. [Required], [MaxLength]).

```
using Microsoft.EntityFrameworkCore;
using ParkingSystem.Domain.Entities;
```

```
namespace ParkingSystem.Infrastructure.Data;
```

```
public class AppDbContext : DbContext
{
    // A Set<T>() metódussal elkerülhető a nullable figyelmeztetés
    public DbSet<ParkingSpace> ParkingSpaces => Set<ParkingSpace>();
    public DbSet<Reservation> Reservations => Set<Reservation>();

    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
    {
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // ParkingSpace konfigurációk
        modelBuilder.Entity<ParkingSpace>()
            .HasKey(p => p.Id);

        // Reservation konfigurációk és kapcsolatok
        modelBuilder.Entity<Reservation>()
            .HasKey(r => r.Id);

        modelBuilder.Entity<Reservation>()
            .Property(r => r.ApplicantName)
            .IsRequired()
            .HasMaxLength(150);

        // 1:N (Egy a többhöz) kapcsolat konfigurálása
        modelBuilder.Entity<Reservation>()
            .HasOne(r => r.ParkingSpace)
            .WithMany(p => p.Reservations)
            .HasForeignKey(r => r.ParkingSpaceId)
            .OnDelete(DeleteBehavior.Cascade); // Ha egy parkolóhely törlődik, a hozzá tartozó foglalások is
    }
}
```

User prompt: A projekt pontos neve ParkingReservation, nem pedig ParkingSystem. Kérlek, írd újra az előző kódokat ezzel a helyes namespace-szel, hogy egy az egyben be tudjam másolni őket. A jövőben is ezt a nevet használd a projekt hivatkozásainál.

Response: Rendben, elnézést kérek a pontatlanságért. Megjegyeztem a **ParkingReservation** projektnevet.

Itt vannak a frissített kódok a megfelelő névterekkel (namespace), így most már egy az egyben bemásolhatod őket a megfelelő fájlokba. Természetesen továbbra is figyeltem arra, hogy a var kulcsszót ne használjam.

ParkingSpaceType.cs

(Mappa: Domain/Enums/)

```
namespace ParkingReservation.Domain.Enums;

public enum ParkingSpaceType
{
    Standard = 0,
    Accessible = 1 // Mozgássérült
}
```

ParkingSpace.cs

(Mappa: Domain/Entities/)

```
using System.Collections.Generic;
using ParkingReservation.Domain.Enums;

namespace ParkingReservation.Domain.Entities;

public class ParkingSpace
{
    public int Id { get; init; }
    public ParkingSpaceType Type { get; set; }

    // Navigációs tulajdonság a foglalások eléréséhez
    // Kezdeti érték adása megakadályozza a NullReferenceException-t
    public ICollection<Reservation> Reservations { get; set; } = new List<Reservation>();
}
```

Reservation.cs

(Mappa: Domain/Entities/)

```
using System;
```

```

namespace ParkingReservation.Domain.Entities;

public class Reservation
{
    public int Id { get; init; }

    // Idegen kulcs
    public int ParkingSpaceId { get; set; }

    // Navigációs tulajdonság (lehet null, amíg az EF be nem tölti)
    public ParkingSpace? ParkingSpace { get; set; }

    // DateTimeOffset használata a pontos idő/időzóna miatt
    public DateTimeOffset StartTime { get; set; }
    public DateTimeOffset EndTime { get; set; }

    // A 'required' biztosítja, hogy a példányosításkor meg kell adni a nevet
    public required string ApplicantName { get; set; }
}

```

AppDbContext.cs

(Mappa: Infrastructure/Data/)

```

using Microsoft.EntityFrameworkCore;
using ParkingReservation.Domain.Entities;

namespace ParkingReservation.Infrastructure.Data;

public class AppDbContext : DbContext
{
    // A Set<T>() metódussal elkerülhető a nullable figyelmeztetés
    public DbSet<ParkingSpace> ParkingSpaces => Set<ParkingSpace>();
    public DbSet<Reservation> Reservations => Set<Reservation>();

    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
    {
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // ParkingSpace konfigurációk
        modelBuilder.Entity<ParkingSpace>()
            .HasKey(p => p.Id);

        // Reservation konfigurációk és kapcsolatok
        modelBuilder.Entity<Reservation>()
            .HasKey(r => r.Id);
    }
}

```

```

modelBuilder.Entity<Reservation>()
    .Property(r => r.ApplicantName)
    .IsRequired()
    .HasMaxLength(150);

// 1:N (Egy a többhöz) kapcsolat konfigurálása
modelBuilder.Entity<Reservation>()
    .HasOne(r => r.ParkingSpace)
    .WithMany(p => p.Reservations)
    .HasForeignKey(r => r.ParkingSpaceId)
    .OnDelete(DeleteBehavior.Cascade); // Ha egy parkolóhely törlődik, a hozzá tartozó foglalások is
}
}

```

User prompt: Az alap modellek és a DbContext a helyükön vannak, és feltelepítettem a szükséges EF Core SQL Server csomagokat is. Kérlek, mutasd meg, hogyan regisztráljam az AppDbContext-et a Program.cs fájlban. Ezen kívül írd egy példa Connection Stringet is az appsettings.json-be, ami egy standard SQL Serverhez (például a jövőben egy Dockerben futó adatbázishoz) csatlakozik 'localhost' címen, 'SA' felhasználóval.

Response: Kiváló, haladjunk tovább. A kontextus regisztrációja az ASP.NET Core beépített dependency injection (DI) konténerébe történik a Program.cs fájlban. Emellett be kell állítanunk a kapcsolódási karakterláncot (connection string) is.

Mivel kérted, hogy kerüljem a var kulcsszót, a Program.cs-ben minden típusnál explicit megadom az osztályt vagy interfészt.

1. appsettings.json beállítása

Az adatbázis kapcsolat adatait a legjobb gyakorlat szerint a konfigurációs fájlban tároljuk. Egy helyi, például Dockerből futtatott SQL Server esetén (ahol a felhasználó SA) így érdemes felépíteni a connection stringet.

Fontos: Az SQL Server alapértelmezetten megköveteli az erős jelszavakat az SA fiókhhoz, illetve helyi fejlesztésnél gyakran szükséges a TrustServerCertificate=True beállítás a fejlesztői SSL tanúsítványok miatt.

```

{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft.AspNetCore": "Warning",
      "Microsoft.EntityFrameworkCore.Database.Command": "Information"
    }
  },
  "ConnectionStrings": {
    "DefaultConnection": "Server=localhost,1433;Database=ParkingReservationDb;UserId=SA;Password=StrongPassword123!;TrustServerCertificate=True;MultipleActiveResultSets=true;"
  }
}

```

```
},  
"AllowedHosts": ""  
}
```

(Tipp: A Microsoft.EntityFrameworkCore.Database.Command naplózási szint beállítása fejlesztés alatt nagyon hasznos, mert így a konzolon látni fogod a generált SQL lekérdezéseket.)

2. Program.cs módosítása

A modern .NET Web API sablonok "Top-level statements" formátumot használnak. Itt olvassuk be a konfigurációt, és regisztráljuk az AppDbContext-et. Iparági best practice, hogy ha nem találjuk a connection stringet, azonnal dobunk egy kivételt (Fail-Fast elv), ahelyett, hogy később futásidőben omlana össze az alkalmazás.

```
using System;  
using Microsoft.AspNetCore.Builder;  
using Microsoft.Extensions.Configuration;  
using Microsoft.Extensions.DependencyInjection;  
using Microsoft.Extensions.Hosting;  
using Microsoft.EntityFrameworkCore;  
using ParkingReservation.Infrastructure.Data;
```

```
WebApplicationBuilder builder = WebApplication.CreateBuilder(args);
```

```
// --- Szolgáltatások (Services) regisztrációja ---
```

```
builder.Services.AddControllers();  
builder.Services.AddEndpointsApiExplorer();  
builder.Services.AddSwaggerGen();
```

```
// Csatlakozási karakterlánc beolvasása biztonságosan
```

```
string connectionString = builder.Configuration.GetConnectionString("DefaultConnection")  
    ?? throw new InvalidOperationException("A 'DefaultConnection' connection string nem található az  
appsettings.json fájlban.");
```

```
// AppDbContext regisztrációja a DI konténerben
```

```
builder.Services.AddDbContext<AppDbContext>(options =>  
{  
    options.UseSqlServer(connectionString);  
});
```

```
WebApplication app = builder.Build();
```

```
// --- HTTP kérés folyamat (Pipeline) konfigurálása ---
```

```
if (app.Environment.IsDevelopment())  
{  
    app.UseSwagger();  
    app.UseSwaggerUI();  
}
```

```
app.UseHttpsRedirection();  
  
app.UseAuthorization();  
  
app.MapControllers();  
  
app.Run();
```

Következő lépések

Miután ezeket beállítottad, a Package Manager Console-ban (vagy terminálban) elkészítheted az első migrációt, és létrehozhatod az adatbázist. Mivel egyetlen projektből áll a megoldás, az alábbi parancsokat a projekt gyökérmappájában futtathatod:

.NET CLI használatával:

```
dotnet ef migrations add InitialCreate  
dotnet ef database update
```

Ezzel a ParkingReservation projekt adatbázis rétege készen áll a használatra, és a kontrollerekben (vagy szervizekben) már injektálható is az ApplicationDbContext konstruktoron keresztül.

User prompt: A kód hibátlanul fordul, és a dotnet ef migrations add InitialCreate is létrejött. Viszont a dotnet ef database update futtatásakor timeout hibát kapok: "A network-related or instance-specific error occurred...". Ez azért van, mert még nem fut az SQL Server a gépemem. Mivel Dockerrel szeretném futtatni az adatbázist, kérlek, írd meg nekem egy pontos docker run parancsot, amivel el tudok indítani egy fejlesztői Microsoft SQL Server konténert a háttérben. A parancs paraméterei pontosan illeszkedjenek ahhoz a Connection Stringhez, amit az előbb adtál (1433-as port, SA felhasználó, és a "StrongPassword123!" jelszó). Kérlek, csak magát a parancsot add meg, és röviden írd le, hogyan futtassam utána újra a migrációt.

Response: docker run -e "ACCEPT_EULA=Y" -e "MSSQL_SA_PASSWORD=StrongPassword123!" -p 1433:1433 --name parking-sql -d mcr.microsoft.com/mssql/server:2022-latest

Miután a konténer elindult (érdemes 5-10 másodpercet várni, amíg az SQL Server motor teljesen feláll a háttérben), futtasd újra az alábbi parancsot a projekt mappájában:

```
dotnet ef database update
```

User prompt: Most készítsunk service réteget és kontrollereket. Szükségünk van egy parkolóhelyeket kezelő és egy foglalásokat kezelő vezérlőre, valamint a mögöttük álló üzleti logikára, amely kezeli a parkolóhelyek rögzítését, listázását, a foglalási kérések felvételét ütközésvizsgálattal, a foglalások lekérdezését, valamint a

lemondást is.

Response: A senior .NET fejlesztői best practiceket követve a service réteget és a kontrollereket DTO-k (Data Transfer Object) használatával választjuk szét a domain modellektől, így biztosítva a tiszta architektúrát és a biztonságos adatkezelést.

A foglalásoknál az ütközésvizsgálat (overlap detection) az adatbázis szintjén ellenőrzi, hogy a megadott időintervallum nem fedésben van-e egy már létező foglalással az adott parkolóhelyen.

A kódrészletekben szigorúan **betartom a var kulcsszó mellőzését**, és mindenhol explicit módon megadom a típusokat.

1. DTO-k (Adatátviteli objektumok)

(Mappa: Application/DTOs/)

```
using System;
using ParkingReservation.Domain.Enums;

namespace ParkingReservation.Application.DTOs;

public class CreateParkingSpaceDto
{
    public ParkingSpaceType Type { get; set; }
}

public class ParkingSpaceDto
{
    public int Id { get; init; }
    public ParkingSpaceType Type { get; set; }
}

public class CreateReservationDto
{
    public int ParkingSpaceId { get; set; }
    public DateTimeOffset StartTime { get; set; }
    public DateTimeOffset EndTime { get; set; }
    public required string ApplicantName { get; set; }
}

public class ReservationDto
{
    public int Id { get; init; }
    public int ParkingSpaceId { get; set; }
    public DateTimeOffset StartTime { get; set; }
    public DateTimeOffset EndTime { get; set; }
    public string ApplicantName { get; set; } = string.Empty;
}
```

2. Service Réteg (Üzleti logika)

(Mappa: Application/Services/)

Parkolóhely szerviz

```
using System.Collections.Generic;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using ParkingReservation.Application.DTOs;
using ParkingReservation.Domain.Entities;
using ParkingReservation.Infrastructure.Data;

namespace ParkingReservation.Application.Services;

public interface IParkingSpaceService
{
    Task<List<ParkingSpaceDto>> GetAllAsync();
    Task<ParkingSpaceDto> CreateAsync(CreateParkingSpaceDto dto);
}

public class ParkingSpaceService : IParkingSpaceService
{
    private readonly AppDbContext _context;

    public ParkingSpaceService(AppDbContext context)
    {
        _context = context;
    }

    public async Task<List<ParkingSpaceDto>> GetAllAsync()
    {
        List<ParkingSpace> spaces = await _context.ParkingSpaces.ToListAsync();
        List<ParkingSpaceDto> result = new List<ParkingSpaceDto>();

        foreach (ParkingSpace space in spaces)
        {
            result.Add(new ParkingSpaceDto
            {
                Id = space.Id,
                Type = space.Type
            });
        }

        return result;
    }

    public async Task<ParkingSpaceDto> CreateAsync(CreateParkingSpaceDto dto)
    {
        ParkingSpace space = new ParkingSpace
        {

```

```

        Type = dto.Type
    };

    _context.ParkingSpaces.Add(space);
    await _context.SaveChangesAsync();

    ParkingSpaceDto result = new ParkingSpaceDto
    {
        Id = space.Id,
        Type = space.Type
    };

    return result;
}
}

```

Foglalás szerviz (ütközésvizsgálattal)

```

using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using ParkingReservation.Application.DTOs;
using ParkingReservation.Domain.Entities;
using ParkingReservation.Infrastructure.Data;

namespace ParkingReservation.Application.Services;

public interface IReservationService
{
    Task<List<ReservationDto>> GetAllAsync();
    Task<ReservationDto?> CreateAsync(CreateReservationDto dto);
    Task<bool> DeleteAsync(int id);
}

public class ReservationService : IReservationService
{
    private readonly AppDbContext _context;

    public ReservationService(AppDbContext context)
    {
        _context = context;
    }

    public async Task<List<ReservationDto>> GetAllAsync()
    {
        List<Reservation> reservations = await _context.Reservations.ToListAsync();
        List<ReservationDto> result = new List<ReservationDto>();

        foreach (Reservation reservation in reservations)
        {

```

```

        result.Add(new ReservationDto
        {
            Id = reservation.Id,
            ParkingSpaceId = reservation.ParkingSpaceId,
            StartTime = reservation.StartTime,
            EndTime = reservation.EndTime,
            ApplicantName = reservation.ApplicantName
        });
    }

    return result;
}

public async Task<ReservationDto?> CreateAsync(CreateReservationDto dto)
{
    // Ellenőrzés: Létezik-e a megadott parkolóhely
    bool spaceExists = await _context.ParkingSpaces.AnyAsync(p => p.Id == dto.ParkingSpaceId);
    if (!spaceExists)
    {
        return null;
    }

    // Ütközésvizsgálat: Megvizsgáljuk, hogy van-e már átfedő foglalás ugyanerre a helyre
    // Két intervallum [StartA, EndA] és [StartB, EndB] akkor ütközik, ha: StartA < EndB ÉS StartB < EndA
    bool hasConflict = await _context.Reservations
        .AnyAsync(r => r.ParkingSpaceId == dto.ParkingSpaceId &&
            r.StartTime < dto.EndTime &&
            dto.StartTime < r.EndTime);

    if (hasConflict)
    {
        throw new InvalidOperationException("A megadott időpontban a parkolóhely már foglalt.");
    }

    Reservation reservation = new Reservation
    {
        ParkingSpaceId = dto.ParkingSpaceId,
        StartTime = dto.StartTime,
        EndTime = dto.EndTime,
        ApplicantName = dto.ApplicantName
    };

    _context.Reservations.Add(reservation);
    await _context.SaveChangesAsync();

    ReservationDto result = new ReservationDto
    {
        Id = reservation.Id,
        ParkingSpaceId = reservation.ParkingSpaceId,
        StartTime = reservation.StartTime,
        EndTime = reservation.EndTime,
        ApplicantName = reservation.ApplicantName
    }

```

```

};

return result;
}

public async Task<bool> DeleteAsync(int id)
{
    Reservation? reservation = await _context.Reservations.FindAsync(id);
    if (reservation == null)
    {
        return false;
    }

    _context.Reservations.Remove(reservation);
    await _context.SaveChangesAsync();
    return true;
}
}

```

3. Kontrollerek

(Mappa: Controllers/)

ParkingSpacesController

```

using System.Collections.Generic;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using ParkingReservation.Application.DTOs;
using ParkingReservation.Application.Services;

namespace ParkingReservation.Controllers;

[ApiController]
[Route("api/[controller]")]
public class ParkingSpacesController : ControllerBase
{
    private readonly IParkingSpaceService _parkingSpaceService;

    public ParkingSpacesController(IParkingSpaceService parkingSpaceService)
    {
        _parkingSpaceService = parkingSpaceService;
    }

    [HttpGet]
    public async Task<ActionResult<List<ParkingSpaceDto>>> GetAll()
    {
        List<ParkingSpaceDto> spaces = await _parkingSpaceService.GetAllAsync();
        return Ok(spaces);
    }
}

```

```

[HttpPost]
public async Task<ActionResult<ParkingSpaceDto>> Create([FromBody] CreateParkingSpaceDto dto)
{
    ParkingSpaceDto createdSpace = await _parkingSpaceService.CreateAsync(dto);
    return CreatedAtAction(nameof(GetAll), new { id = createdSpace.Id }, createdSpace);
}
}

```

ReservationsController

```

using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using ParkingReservation.Application.DTOs;
using ParkingReservation.Application.Services;

```

```

namespace ParkingReservation.Controllers;

```

```

[ApiController]
[Route("api/[controller]")]
public class ReservationsController : ControllerBase
{

```

```

    private readonly IReservationService _reservationService;

```

```

    public ReservationsController(IReservationService reservationService)
    {
        _reservationService = reservationService;
    }

```

```

[HttpGet]
public async Task<ActionResult<List<ReservationDto>>> GetAll()
{
    List<ReservationDto> reservations = await _reservationService.GetAllAsync();
    return Ok(reservations);
}

```

```

[HttpPost]
public async Task<ActionResult<ReservationDto>> Create([FromBody] CreateReservationDto dto)
{
    try
    {
        ReservationDto? createdReservation = await _reservationService.CreateAsync(dto);
        if (createdReservation == null)
        {
            return BadRequest("A megadott parkolóhely nem létezik.");
        }
        return CreatedAtAction(nameof(GetAll), new { id = createdReservation.Id }, createdReservation);
    }
    catch (InvalidOperationException ex)

```

```

    {
        return Conflict(ex.Message);
    }
}

[HttpDelete("{id}")]
public async Task<IActionResult> Delete(int id)
{
    bool deleted = await _reservationService.DeleteAsync(id);
    if (!deleted)
    {
        return NotFound();
    }
    return NoContent();
}
}

```

4. Szolgáltatások regisztrációja a Program.cs-ben

Ahhoz, hogy a kontrollerek be tudják injektálni a szervizeket, regisztrálni kell őket a DI konténerbe a Program.cs fájlban (közvetlenül az ApplicationDbContext regisztrációja után):

```

builder.Services.AddScoped<IParkingSpaceService, ParkingSpaceService>();
builder.Services.AddScoped<IReservationService, ReservationService>();

```

User prompt: Kérlek módosítsd a kódot a következők szerint: Minden DTO osztály kerüljön külön fájlba az Application/DTOs mappán belül. Egészítsd ki a rendszert egy olyan végponttal, amellyel le lehet kérdezni egy adott parkolóhelyhez tartozó foglalásokat is.

Response: Íme a módosított kód, amelyben minden DTO osztály külön fájlba került az Application/DTOs/ mappán belül, és kiegészítettem a rendszert egy új végponttal, amellyel egy adott parkolóhelyhez tartozó foglalások is lekérdezhetők.

A szigorú típusosság megőrzése érdekében **a var kulcsszót most sem használtam**.

1. DTO-k (Külön fájlokban az Application/DTOs mappán belül)

CreateParkingSpaceDto.cs

```

using ParkingReservation.Domain.Enums;

namespace ParkingReservation.Application.DTOs;

public class CreateParkingSpaceDto
{
    public ParkingSpaceType Type { get; set; }
}

```

ParkingSpaceDto.cs

```
using ParkingReservation.Domain.Enums;

namespace ParkingReservation.Application.DTOs;

public class ParkingSpaceDto
{
    public int Id { get; init; }
    public ParkingSpaceType Type { get; set; }
}
```

CreateReservationDto.cs

```
using System;

namespace ParkingReservation.Application.DTOs;

public class CreateReservationDto
{
    public int ParkingSpaceId { get; set; }
    public DateTimeOffset StartTime { get; set; }
    public DateTimeOffset EndTime { get; set; }
    public required string ApplicantName { get; set; }
}
```

ReservationDto.cs

```
using System;

namespace ParkingReservation.Application.DTOs;

public class ReservationDto
{
    public int Id { get; init; }
    public int ParkingSpaceId { get; set; }
    public DateTimeOffset StartTime { get; set; }
    public DateTimeOffset EndTime { get; set; }
    public string ApplicantName { get; set; } = string.Empty;
}
```

2. Service Réteg módosítása (Új lekérdezési metódus)

Az IReservationService interfészt és a ReservationService osztályt ki kell egészíteni a parkolóhely szerinti lekérdezéssel.

IReservationService interfész kiegészítés

```
using System.Collections.Generic;
using System.Threading.Tasks;
using ParkingReservation.Application.DTOs;

namespace ParkingReservation.Application.Services;

public interface IReservationService
{
    Task<List<ReservationDto>> GetAllAsync();
    Task<List<ReservationDto>> GetByParkingSpaceIdAsync(int parkingSpaceId);
    Task<ReservationDto?> CreateAsync(CreateReservationDto dto);
    Task<bool> DeleteAsync(int id);
}
```

ReservationService megvalósítás (frissítve)

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using ParkingReservation.Application.DTOs;
using ParkingReservation.Domain.Entities;
using ParkingReservation.Infrastructure.Data;

namespace ParkingReservation.Application.Services;

public class ReservationService : IReservationService
{
    private readonly AppDbContext _context;

    public ReservationService(AppDbContext context)
    {
        _context = context;
    }

    public async Task<List<ReservationDto>> GetAllAsync()
    {
        List<Reservation> reservations = await _context.Reservations.ToListAsync();
        List<ReservationDto> result = new List<ReservationDto>();

        foreach (Reservation reservation in reservations)
        {
            result.Add(new ReservationDto
            {
                Id = reservation.Id,
                ParkingSpaceId = reservation.ParkingSpaceId,
                StartTime = reservation.StartTime,
                EndTime = reservation.EndTime,
            });
        }
    }
}
```

```

        ApplicantName = reservation.ApplicantName
    });
}

return result;
}

public async Task<List<ReservationDto>> GetByParkingSpaceIdAsync(int parkingSpaceId)
{
    List<Reservation> reservations = await _context.Reservations
        .Where(r => r.ParkingSpaceId == parkingSpaceId)
        .ToListAsync();

    List<ReservationDto> result = new List<ReservationDto>();

    foreach (Reservation reservation in reservations)
    {
        result.Add(new ReservationDto
        {
            Id = reservation.Id,
            ParkingSpaceId = reservation.ParkingSpaceId,
            StartTime = reservation.StartTime,
            EndTime = reservation.EndTime,
            ApplicantName = reservation.ApplicantName
        });
    }

    return result;
}

public async Task<ReservationDto?> CreateAsync(CreateReservationDto dto)
{
    bool spaceExists = await _context.ParkingSpaces.AnyAsync(p => p.Id == dto.ParkingSpaceId);
    if (!spaceExists)
    {
        return null;
    }

    bool hasConflict = await _context.Reservations
        .AnyAsync(r => r.ParkingSpaceId == dto.ParkingSpaceId &&
            r.StartTime < dto.EndTime &&
            dto.StartTime < r.EndTime);

    if (hasConflict)
    {
        throw new InvalidOperationException("A megadott időpontban a parkolóhely már foglalt.");
    }

    Reservation reservation = new Reservation
    {
        ParkingSpaceId = dto.ParkingSpaceId,
        StartTime = dto.StartTime,

```

```

        EndTime = dto.EndTime,
        ApplicantName = dto.ApplicantName
    };

    _context.Reservations.Add(reservation);
    await _context.SaveChangesAsync();

    ReservationDto result = new ReservationDto
    {
        Id = reservation.Id,
        ParkingSpaceId = reservation.ParkingSpaceId,
        StartTime = reservation.StartTime,
        EndTime = reservation.EndTime,
        ApplicantName = reservation.ApplicantName
    };

    return result;
}

public async Task<bool> DeleteAsync(int id)
{
    Reservation? reservation = await _context.Reservations.FindAsync(id);
    if (reservation == null)
    {
        return false;
    }

    _context.Reservations.Remove(reservation);
    await _context.SaveChangesAsync();
    return true;
}
}

```

3. Kontroller módosítása az új végponthoz

A REST-elveket követve logikus, hogy a parkolóhelyekhez tartozó foglalásokat a ParkingSpacesController-en keresztül érjük el egy alútvonalon (GET api/parkingspaces/{id}/reservations). Ehhez be kell injektálni az IReservationService-t is a vezérlőbe.

ParkingSpacesController.cs (frissítve)

```

using System.Collections.Generic;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using ParkingReservation.Application.DTOs;
using ParkingReservation.Application.Services;

```

```

namespace ParkingReservation.Controllers;

```

```

[ApiController]

```

```

[Route("api/[controller]")]
public class ParkingSpacesController : ControllerBase
{
    private readonly IParkingSpaceService _parkingSpaceService;
    private readonly IReservationService _reservationService;

    public ParkingSpacesController(
        IParkingSpaceService parkingSpaceService,
        IReservationService reservationService)
    {
        _parkingSpaceService = parkingSpaceService;
        _reservationService = reservationService;
    }

    [HttpGet]
    public async Task<ActionResult<List<ParkingSpaceDto>>> GetAll()
    {
        List<ParkingSpaceDto> spaces = await _parkingSpaceService.GetAllAsync();
        return Ok(spaces);
    }

    [HttpPost]
    public async Task<ActionResult<ParkingSpaceDto>> Create([FromBody] CreateParkingSpaceDto dto)
    {
        ParkingSpaceDto createdSpace = await _parkingSpaceService.CreateAsync(dto);
        return CreatedAtAction(nameof(GetAll), new { id = createdSpace.Id }, createdSpace);
    }

    // Új végpont: adott parkolóhelyhez tartozó foglalások lekérdezése
    [HttpGet("{id}/reservations")]
    public async Task<ActionResult<List<ReservationDto>>> GetReservationsByParkingSpace(int id)
    {
        List<ParkingSpaceDto> spaces = await _parkingSpaceService.GetAllAsync();
        bool spaceExists = false;

        foreach (ParkingSpaceDto space in spaces)
        {
            if (space.Id == id)
            {
                spaceExists = true;
                break;
            }
        }

        if (!spaceExists)
        {
            return NotFound("A megadott parkolóhely nem létezik.");
        }

        List<ReservationDto> reservations = await _reservationService.GetByParkingSpaceIdAsync(id);
        return Ok(reservations);
    }
}

```

```
}
```

User prompt: Az interfészek (pl. IParkingSpaceService) és az azokat megvalósító osztályok (ParkingSpaceService) szintén kerüljenek külön-külön fájlokba az Application/Services mappában.

Response: Természetesen, a tiszta architektúra (Clean Architecture) és a SOLID elvek alapján is ez a helyes megközelítés. Így a kód sokkal átláthatóbb és karbantarthatóbb lesz.

Íme a szétbontott fájlok szerkezete az Application/Services/ mappán belül. Ahogy kérted, **továbbra is szigorúan mellőzöm a var kulcsszó használatát.**

IParkingSpaceService.cs

```
using System.Collections.Generic;
using System.Threading.Tasks;
using ParkingReservation.Application.DTOs;

namespace ParkingReservation.Application.Services;

public interface IParkingSpaceService
{
    Task<List<ParkingSpaceDto>> GetAllAsync();
    Task<ParkingSpaceDto> CreateAsync(CreateParkingSpaceDto dto);
}
```

ParkingSpaceService.cs

```
using System.Collections.Generic;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using ParkingReservation.Application.DTOs;
using ParkingReservation.Domain.Entities;
using ParkingReservation.Infrastructure.Data;

namespace ParkingReservation.Application.Services;

public class ParkingSpaceService : IParkingSpaceService
{
    private readonly AppDbContext _context;

    public ParkingSpaceService(AppDbContext context)
    {
        _context = context;
    }

    public async Task<List<ParkingSpaceDto>> GetAllAsync()
    {
        List<ParkingSpace> spaces = await _context.ParkingSpaces.ToListAsync();
    }
}
```

```

        List<ParkingSpaceDto> result = new List<ParkingSpaceDto>();

        foreach (ParkingSpace space in spaces)
        {
            result.Add(new ParkingSpaceDto
            {
                Id = space.Id,
                Type = space.Type
            });
        }

        return result;
    }

    public async Task<ParkingSpaceDto> CreateAsync(CreateParkingSpaceDto dto)
    {
        ParkingSpace space = new ParkingSpace
        {
            Type = dto.Type
        };

        _context.ParkingSpaces.Add(space);
        await _context.SaveChangesAsync();

        ParkingSpaceDto result = new ParkingSpaceDto
        {
            Id = space.Id,
            Type = space.Type
        };

        return result;
    }
}

```

IReservationService.cs

```

using System.Collections.Generic;
using System.Threading.Tasks;
using ParkingReservation.Application.DTOs;

namespace ParkingReservation.Application.Services;

public interface IReservationService
{
    Task<List<ReservationDto>> GetAllAsync();
    Task<List<ReservationDto>> GetByParkingSpaceIdAsync(int parkingSpaceId);
    Task<ReservationDto?> CreateAsync(CreateReservationDto dto);
    Task<bool> DeleteAsync(int id);
}

```

ReservationService.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using ParkingReservation.Application.DTOs;
using ParkingReservation.Domain.Entities;
using ParkingReservation.Infrastructure.Data;

namespace ParkingReservation.Application.Services;

public class ReservationService : IReservationService
{
    private readonly AppDbContext _context;

    public ReservationService(AppDbContext context)
    {
        _context = context;
    }

    public async Task<List<ReservationDto>> GetAllAsync()
    {
        List<Reservation> reservations = await _context.Reservations.ToListAsync();
        List<ReservationDto> result = new List<ReservationDto>();

        foreach (Reservation reservation in reservations)
        {
            result.Add(new ReservationDto
            {
                Id = reservation.Id,
                ParkingSpaceId = reservation.ParkingSpaceId,
                StartTime = reservation.StartTime,
                EndTime = reservation.EndTime,
                ApplicantName = reservation.ApplicantName
            });
        }

        return result;
    }

    public async Task<List<ReservationDto>> GetByParkingSpaceIdAsync(int parkingSpaceId)
    {
        List<Reservation> reservations = await _context.Reservations
            .Where(r => r.ParkingSpaceId == parkingSpaceId)
            .ToListAsync();

        List<ReservationDto> result = new List<ReservationDto>();

        foreach (Reservation reservation in reservations)
        {
```

```

        result.Add(new ReservationDto
        {
            Id = reservation.Id,
            ParkingSpaceId = reservation.ParkingSpaceId,
            StartTime = reservation.StartTime,
            EndTime = reservation.EndTime,
            ApplicantName = reservation.ApplicantName
        });
    }

    return result;
}

public async Task<ReservationDto?> CreateAsync(CreateReservationDto dto)
{
    bool spaceExists = await _context.ParkingSpaces.AnyAsync(p => p.Id == dto.ParkingSpaceId);
    if (!spaceExists)
    {
        return null;
    }

    bool hasConflict = await _context.Reservations
        .AnyAsync(r => r.ParkingSpaceId == dto.ParkingSpaceId &&
            r.StartTime < dto.EndTime &&
            dto.StartTime < r.EndTime);

    if (hasConflict)
    {
        throw new InvalidOperationException("A megadott időpontban a parkolóhely már foglalt.");
    }

    Reservation reservation = new Reservation
    {
        ParkingSpaceId = dto.ParkingSpaceId,
        StartTime = dto.StartTime,
        EndTime = dto.EndTime,
        ApplicantName = dto.ApplicantName
    };

    _context.Reservations.Add(reservation);
    await _context.SaveChangesAsync();

    ReservationDto result = new ReservationDto
    {
        Id = reservation.Id,
        ParkingSpaceId = reservation.ParkingSpaceId,
        StartTime = reservation.StartTime,
        EndTime = reservation.EndTime,
        ApplicantName = reservation.ApplicantName
    };

    return result;
}

```



```

    }

    public async Task<bool> DeleteAsync(int id)
    {
        Reservation? reservation = await _context.Reservations.FindAsync(id);
        if (reservation == null)
        {
            return false;
        }

        _context.Reservations.Remove(reservation);
        await _context.SaveChangesAsync();
        return true;
    }
}

```

User prompt: Kérlek, módosítsd a projektünket és a meglévő kódokat az alábbi új funkciókkal: 1. Böngésző automatikus indítása: Módosítsd a Properties/launchSettings.json fájlt úgy, hogy a Swagger UI automatikusan megnyíljon az alkalmazás indításakor. 2. Kezdőadatok a meglévő DbContext-ben: Bővítsd ki az AppDbContext osztály OnModelCreating metódusát a HasData metódus használatával. Hozz létre benne néhány alapértelmezett parkolóhelyet, köztük normál és mozgássérült típusokat, és rögzíts hozzájuk induló foglalásokat is. 3. Extra üzleti logika: Bővítsd ki a CreateReservationDto-t egy bool HasSpecialPermit mezővel. A ReservationService foglalási logikáját módosítsd úgy, hogy ha a felhasználó egy mozgássérült parkolóhelyet próbál lefoglalni, a rendszer ellenőrizze ezt a bool mezőt. Ha a hely mozgássérült, de a HasSpecialPermit értéke false, a rendszer dobjon kivételt, és utasítsa el a foglalást.

Response: Örömmel beépítem ezeket az új funkciókat a projektbe. A kéréseid alapján az alábbi pontokon avatkozunk be a kódba:

1. **launchSettings.json:** Beállítjuk a launchBrowser: true és a launchUrl: "swagger" tulajdonságokat, így az API indításakor azonnal láthatod a Swagger felületet.
2. **AppDbContext:** A HasData metódusok segítségével felveszünk statikus alapértelmezett adatokat. *Fontos best practice:* a migrációk miatt a DateTimeOffset értékeket fix, parszolható sztringekből érdemes létrehozni, és nem a DateTimeOffset.Now-t használni, különben minden migráció-készítéskor megváltoznának az értékek.
3. **Üzleti logika (ReservationService & DTO):** A parkolóhely meglétének vizsgálatát módosítjuk úgy, hogy ne csak egy AnyAsync futást végezzen, hanem le is kérdezze a parkolóhely entitást. Így meg tudjuk vizsgálni a Type tulajdonságát, és összevethetjük a DTO-ban érkező új HasSpecialPermit flaggel.

Természetesen **továbbra is mellőzöm a var kulcsszó használatát**, szigorúan tartva a típusosságot.

1. launchSettings.json frissítése

(Mappa: Properties/)

A meglévő fájlod tartalmát az alábbira cserélheted. (Az applicationUrl és a portszámok a te környezetedtől

függhetnek, de a lényeg a launchBrowser és a launchUrl.)

```
{
  "$schema": "https://json.schemastore.org/launchsettings.json",
  "profiles": {
    "http": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "http://localhost:5000",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    },
    "https": {
      "commandName": "Project",
      "dotnetRunMessages": true,
      "launchBrowser": true,
      "launchUrl": "swagger",
      "applicationUrl": "https://localhost:5001;http://localhost:5000",
      "environmentVariables": {
        "ASPNETCORE_ENVIRONMENT": "Development"
      }
    }
  }
}
```

2. DTO módosítása

(Mappa: Application/DTOs/CreateReservationDto.cs)

Hozzáadjuk a logikai (bool) mezőt.

```
using System;

namespace ParkingReservation.Application.DTOs;

public class CreateReservationDto
{
    public int ParkingSpaceId { get; set; }
    public DateTimeOffset StartTime { get; set; }
    public DateTimeOffset EndTime { get; set; }
    public required string ApplicantName { get; set; }

    // Új mező a jogosultság ellenőrzésére
    public bool HasSpecialPermit { get; set; }
}
```

3. ApplicationDbContext módosítása (Kezdőadatok - Seeding)

(Mappa: Infrastructure/Data/AppDbContext.cs)

Kibővítjük az OnModelCreating metódust az adatok feltöltésével.

```
using System;
using Microsoft.EntityFrameworkCore;
using ParkingReservation.Domain.Entities;
using ParkingReservation.Domain.Enums;

namespace ParkingReservation.Infrastructure.Data;

public class AppDbContext : DbContext
{
    public DbSet<ParkingSpace> ParkingSpaces => Set<ParkingSpace>();
    public DbSet<Reservation> Reservations => Set<Reservation>();

    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
    {
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // ParkingSpace konfigurációk
        modelBuilder.Entity<ParkingSpace>()
            .HasKey(p => p.Id);

        // Reservation konfigurációk és kapcsolatok
        modelBuilder.Entity<Reservation>()
            .HasKey(r => r.Id);

        modelBuilder.Entity<Reservation>()
            .Property(r => r.ApplicantName)
            .IsRequired()
            .HasMaxLength(150);

        modelBuilder.Entity<Reservation>()
            .HasOne(r => r.ParkingSpace)
            .WithMany(p => p.Reservations)
            .HasForeignKey(r => r.ParkingSpaceId)
            .OnDelete(DeleteBehavior.Cascade);

        // --- Adatbázis kezdőadatok (Seeding) ---

        modelBuilder.Entity<ParkingSpace>().HasData(
            new ParkingSpace { Id = 1, Type = ParkingSpaceType.Standard },
            new ParkingSpace { Id = 2, Type = ParkingSpaceType.Accessible },
            new ParkingSpace { Id = 3, Type = ParkingSpaceType.Standard }
        );
    }
}
```

```

);

modelBuilder.Entity<Reservation>().HasData(
    new Reservation
    {
        Id = 1,
        ParkingSpaceId = 1,
        ApplicantName = "Kovács Péter",
        StartTime = DateTimeOffset.Parse("2026-09-01T08:00:00+02:00"),
        EndTime = DateTimeOffset.Parse("2026-09-01T12:00:00+02:00")
    },
    new Reservation
    {
        Id = 2,
        ParkingSpaceId = 2,
        ApplicantName = "Nagy Anna",
        StartTime = DateTimeOffset.Parse("2026-09-01T09:00:00+02:00"),
        EndTime = DateTimeOffset.Parse("2026-09-01T14:00:00+02:00")
    }
);
}
}

```

4. Üzleti logika módosítása

(Mappa: *Application/Services/ReservationService.cs*)

Frissítjük a CreateAsync metódust a mozgássérült parkolóhely logikájával. A kód többi része változatlan marad.

```

using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using ParkingReservation.Application.DTOs;
using ParkingReservation.Domain.Entities;
using ParkingReservation.Domain.Enums;
using ParkingReservation.Infrastructure.Data;

namespace ParkingReservation.Application.Services;

public class ReservationService : IReservationService
{
    private readonly AppDbContext _context;

    public ReservationService(AppDbContext context)
    {
        _context = context;
    }
}

```

```

public async Task<List<ReservationDto>> GetAllAsync()
{
    List<Reservation> reservations = await _context.Reservations.ToListAsync();
    List<ReservationDto> result = new List<ReservationDto>();

    foreach (Reservation reservation in reservations)
    {
        result.Add(new ReservationDto
        {
            Id = reservation.Id,
            ParkingSpaceId = reservation.ParkingSpaceId,
            StartTime = reservation.StartTime,
            EndTime = reservation.EndTime,
            ApplicantName = reservation.ApplicantName
        });
    }

    return result;
}

public async Task<List<ReservationDto>> GetByParkingSpaceIdAsync(int parkingSpaceId)
{
    List<Reservation> reservations = await _context.Reservations
        .Where(r => r.ParkingSpaceId == parkingSpaceId)
        .ToListAsync();

    List<ReservationDto> result = new List<ReservationDto>();

    foreach (Reservation reservation in reservations)
    {
        result.Add(new ReservationDto
        {
            Id = reservation.Id,
            ParkingSpaceId = reservation.ParkingSpaceId,
            StartTime = reservation.StartTime,
            EndTime = reservation.EndTime,
            ApplicantName = reservation.ApplicantName
        });
    }

    return result;
}

public async Task<ReservationDto?> CreateAsync(CreateReservationDto dto)
{
    // Változás: AnyAsync helyett le is kérjük a parkolóhelyet, hogy lássuk a típusát
    ParkingSpace? space = await _context.ParkingSpaces.FirstOrDefaultAsync(p => p.Id ==
dto.ParkingSpaceId);

    if (space == null)
    {

```

```

    return null; // A kontroller BadRequest-et ad vissza ez alapján
}

// --- Új üzleti logika: Mozgássérült jogosultság ellenőrzése ---
if (space.Type == ParkingSpaceType.Accessible && dto.HasSpecialPermit == false)
{
    throw new InvalidOperationException("Mozgássérült parkolóhely foglalásához érvényes engedély szükséges (HasSpecialPermit = true).");
}

// Ütközésvizsgálat
bool hasConflict = await _context.Reservations
    .AnyAsync(r => r.ParkingSpaceId == dto.ParkingSpaceId &&
        r.StartTime < dto.EndTime &&
        dto.StartTime < r.EndTime);

if (hasConflict)
{
    throw new InvalidOperationException("A megadott időpontban a parkolóhely már foglalt.");
}

Reservation reservation = new Reservation
{
    ParkingSpaceId = dto.ParkingSpaceId,
    StartTime = dto.StartTime,
    EndTime = dto.EndTime,
    ApplicantName = dto.ApplicantName
};

_context.Reservations.Add(reservation);
await _context.SaveChangesAsync();

ReservationDto result = new ReservationDto
{
    Id = reservation.Id,
    ParkingSpaceId = reservation.ParkingSpaceId,
    StartTime = reservation.StartTime,
    EndTime = reservation.EndTime,
    ApplicantName = reservation.ApplicantName
};

return result;
}

public async Task<bool> DeleteAsync(int id)
{
    Reservation? reservation = await _context.Reservations.FindAsync(id);
    if (reservation == null)
    {
        return false;
    }
}

```

```

        _context.Reservations.Remove(reservation);
        await _context.SaveChangesAsync();
        return true;
    }
}

```

Tipp a fejlesztéshez: Ne felejtse el az új HasData módosítások után létrehozni egy új migrációt (pl. `dotnet ef migrations add AddSeedData`), és frissíteni az adatbázist a `dotnet ef database update` paranccsal, hogy a tesztadatok bekerüljenek az SQL Server konténerbe.

User prompt: Kérlek, módosítsd az AppDbContext osztály OnModelCreating metódusában a Data Seeding logikát. Egy reális parkolóház méreteihez igazodva generálj le 100 darab parkolóhelyet ciklusok segítségével. Az elosztás legyen a következő: 1-80 ID: Normál parkolóhely 81-90 ID: Mozgássérült parkolóhely 91-100 ID: Elektromos parkolóhely Miután a parkolóhelyek listája elkészült, generálj le egy listát 50 darab különböző foglalással is, amelyek hivatkoznak ezekre a parkolóhelyekre az elkövetkezendő napokban. A listákat add át a HasData metódusoknak.

Response: Örömmel megírom neked ezt a frissítést. Mivel egy új parkolóhely-típust (Elektromos) is kértél, először ki kell bővítenünk a ParkingSpaceType enumot.

Ezután az AppDbContext osztály OnModelCreating metódusában listákat (`List<T>`) fogunk feltölteni ciklusok segítségével, majd ezeket adjuk át a HasData metódusnak. A foglalásokhoz egy fix bázisdátumból indulunk ki (hogy a migrációk konzisztensek maradjanak), és ahhoz adunk hozzá napokat és órákat, így az elkövetkezendő napokra szórjuk szét az 50 foglalást.

Természetesen **továbbra is szigorúan mellőzöm a var kulcsszót.**

1. Enum bővítése (Elektromos típus)

(Mappa: *Domain/Enums/ParkingSpaceType.cs*)

```

namespace ParkingReservation.Domain.Enums;

public enum ParkingSpaceType
{
    Standard = 0,
    Accessible = 1,
    Electric = 2 // Új elektromos típus
}

```

2. AppDbContext módosítása (Adatgenerálás ciklusokkal)

(Mappa: *Infrastructure/Data/AppDbContext.cs*)

```

using System;
using System.Collections.Generic;

```

```

using Microsoft.EntityFrameworkCore;
using ParkingReservation.Domain.Entities;
using ParkingReservation.Domain.Enums;

namespace ParkingReservation.Infrastructure.Data;

public class AppDbContext : DbContext
{
    public DbSet<ParkingSpace> ParkingSpaces => Set<ParkingSpace>();
    public DbSet<Reservation> Reservations => Set<Reservation>();

    public AppDbContext(DbContextOptions<AppDbContext> options) : base(options)
    {
    }

    protected override void OnModelCreating(ModelBuilder modelBuilder)
    {
        base.OnModelCreating(modelBuilder);

        // ParkingSpace konfigurációk
        modelBuilder.Entity<ParkingSpace>()
            .HasKey(p => p.Id);

        // Reservation konfigurációk és kapcsolatok
        modelBuilder.Entity<Reservation>()
            .HasKey(r => r.Id);

        modelBuilder.Entity<Reservation>()
            .Property(r => r.ApplicantName)
            .IsRequired()
            .HasMaxLength(150);

        modelBuilder.Entity<Reservation>()
            .HasOne(r => r.ParkingSpace)
            .WithMany(p => p.Reservations)
            .HasForeignKey(r => r.ParkingSpaceId)
            .OnDelete(DeleteBehavior.Cascade);

        // --- Adatbázis kezdőadatok generálása (Seeding) ---

        List<ParkingSpace> parkingSpaces = new List<ParkingSpace>();

        // 1-80: Normál parkolóhely
        for (int i = 1; i <= 80; i++)
        {
            parkingSpaces.Add(new ParkingSpace { Id = i, Type = ParkingSpaceType.Standard });
        }

        // 81-90: Mozgássérült parkolóhely
        for (int i = 81; i <= 90; i++)
        {
            parkingSpaces.Add(new ParkingSpace { Id = i, Type = ParkingSpaceType.Accessible });
        }
    }
}

```



```

    }

    // 91-100: Elektromos parkolóhely
    for (int i = 91; i <= 100; i++)
    {
        parkingSpaces.Add(new ParkingSpace { Id = i, Type = ParkingSpaceType.Electric });
    }

    // Parkolóhelyek átadása a HasData-nak
    modelBuilder.Entity<ParkingSpace>().HasData(parkingSpaces);

    List<Reservation> reservations = new List<Reservation>();

    // Egy fix bázisdátum a jövőben, hogy a migráció futtatásakor mindig ugyanazt az eredményt kapjuk
    DateTimeOffset baseDate = DateTimeOffset.Parse("2026-08-10T08:00:00+02:00");

    // 50 darab foglalás generálása
    for (int i = 1; i <= 50; i++)
    {
        // A napokat 0 és 9 között variáljuk, az órákat pedig 0 és 4 között tolásoljuk a bázisdátumhoz képest
        int dayOffset = i % 10;
        int hourOffset = i % 5;

        reservations.Add(new Reservation
        {
            Id = i,
            // Minden foglalás egy külön parkolóhelyre kerül (1-től 50-ig), így biztosan nem lesz ütközés
            ParkingSpaceId = i,
            ApplicantName = "Teszt Felhasználó " + i.ToString(),
            StartTime = baseDate.AddDays(dayOffset).AddHours(hourOffset),
            // Minden foglalás 2 óra hosszúságú
            EndTime = baseDate.AddDays(dayOffset).AddHours(hourOffset + 2)
        });
    }

    // Foglalások átadása a HasData-nak
    modelBuilder.Entity<Reservation>().HasData(reservations);
}
}

```

Következő lépés

Mivel módosítottad a kezdőadatokat (Seeding logic) és egy enum is bővült, az Entity Framework-nek szüksége lesz egy új migrációra a változások adatbázisba vezetéséhez. Futtasd le az alábbi parancsokat a projektged gyökérmappájában:

```

dotnet ef migrations add UpdateSeedDataWith100Spaces
dotnet ef database update

```

User prompt: Bővítsd a CreateReservationDto-t egy HasElectricVehicle bool mezővel. A ReservationService-ben oldd meg, hogy elektromos parkolóhelyet csak akkor lehessen lefoglalni, ha ez a mező igaz. Ha hamis, a rendszer dobjon kivételt.

Response: Készséggel kiegészítem a kéréseid alapján a kódot. Az új HasElectricVehicle mező hozzáadásával és a megfelelő ellenőrző logikával az elektromos parkolóhelyek is védelem alá kerülnek.

Természetesen **továbbra is szigorúan kerülöm a var kulcsszó használatát.**

1. DTO módosítása

(Mappa: Application/DTOs/CreateReservationDto.cs)

Hozzáadjuk az új logikai mezőt a DTO-hoz.

```
using System;

namespace ParkingReservation.Application.DTOs;

public class CreateReservationDto
{
    public int ParkingSpaceId { get; set; }
    public DateTimeOffset StartTime { get; set; }
    public DateTimeOffset EndTime { get; set; }
    public required string ApplicantName { get; set; }

    // Mozcássérült jogosultság ellenőrzésére
    public bool HasSpecialPermit { get; set; }

    // Új mező az elektromos járművek ellenőrzésére
    public bool HasElectricVehicle { get; set; }
}
```

2. Üzleti logika módosítása

(Mappa: Application/Services/ReservationService.cs)

Kibővítjük a CreateAsync metódust az elektromos parkolóhelyek vizsgálatával. A fájl többi része változatlan.

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using ParkingReservation.Application.DTOs;
using ParkingReservation.Domain.Entities;
using ParkingReservation.Domain.Enums;
using ParkingReservation.Infrastructure.Data;
```

```

namespace ParkingReservation.Application.Services;

public class ReservationService : IReservationService
{
    private readonly AppDbContext _context;

    public ReservationService(AppDbContext context)
    {
        _context = context;
    }

    public async Task<List<ReservationDto>> GetAllAsync()
    {
        List<Reservation> reservations = await _context.Reservations.ToListAsync();
        List<ReservationDto> result = new List<ReservationDto>();

        foreach (Reservation reservation in reservations)
        {
            result.Add(new ReservationDto
            {
                Id = reservation.Id,
                ParkingSpaceId = reservation.ParkingSpaceId,
                StartTime = reservation.StartTime,
                EndTime = reservation.EndTime,
                ApplicantName = reservation.ApplicantName
            });
        }

        return result;
    }

    public async Task<List<ReservationDto>> GetByParkingSpaceIdAsync(int parkingSpaceId)
    {
        List<Reservation> reservations = await _context.Reservations
            .Where(r => r.ParkingSpaceId == parkingSpaceId)
            .ToListAsync();

        List<ReservationDto> result = new List<ReservationDto>();

        foreach (Reservation reservation in reservations)
        {
            result.Add(new ReservationDto
            {
                Id = reservation.Id,
                ParkingSpaceId = reservation.ParkingSpaceId,
                StartTime = reservation.StartTime,
                EndTime = reservation.EndTime,
                ApplicantName = reservation.ApplicantName
            });
        }
    }
}

```

```

    return result;
}

public async Task<ReservationDto?> CreateAsync(CreateReservationDto dto)
{
    ParkingSpace? space = await _context.ParkingSpaces.FirstOrDefaultAsync(p => p.Id ==
dto.ParkingSpaceId);

    if (space == null)
    {
        return null;
    }

    // Mozgássérült jogosultság ellenőrzése
    if (space.Type == ParkingSpaceType.Accessible && dto.HasSpecialPermit == false)
    {
        throw new InvalidOperationException("Mozgássérült parkolóhely foglalásához érvényes engedély
szükséges (HasSpecialPermit = true).");
    }

    // --- Új üzleti logika: Elektromos jármű ellenőrzése ---
    if (space.Type == ParkingSpaceType.Electric && dto.HasElectricVehicle == false)
    {
        throw new InvalidOperationException("Elektromos parkolóhely foglalásához elektromos jármű
szükséges (HasElectricVehicle = true).");
    }

    // Ütközésvizsgálat
    bool hasConflict = await _context.Reservations
        .AnyAsync(r => r.ParkingSpaceId == dto.ParkingSpaceId &&
            r.StartTime < dto.EndTime &&
            dto.StartTime < r.EndTime);

    if (hasConflict)
    {
        throw new InvalidOperationException("A megadott időpontban a parkolóhely már foglalt.");
    }

    Reservation reservation = new Reservation
    {
        ParkingSpaceId = dto.ParkingSpaceId,
        StartTime = dto.StartTime,
        EndTime = dto.EndTime,
        ApplicantName = dto.ApplicantName
    };

    _context.Reservations.Add(reservation);
    await _context.SaveChangesAsync();

    ReservationDto result = new ReservationDto
    {
        Id = reservation.Id,

```

```

        ParkingSpaceId = reservation.ParkingSpaceId,
        StartTime = reservation.StartTime,
        EndTime = reservation.EndTime,
        ApplicantName = reservation.ApplicantName
    };

    return result;
}

public async Task<bool> DeleteAsync(int id)
{
    Reservation? reservation = await _context.Reservations.FindAsync(id);
    if (reservation == null)
    {
        return false;
    }

    _context.Reservations.Remove(reservation);
    await _context.SaveChangesAsync();
    return true;
}
}

```

User prompt: Kérlek, írd át a fenti xUnit teszteket a projekt összes funkciójának lefedésére. A tesztprojektben hozd létre az alábbi tesztesztállományokat: 1. ReservationServiceTests Használd a Microsoft.EntityFrameworkCore.InMemory adatbázist. Teszteld le a teljes üzleti logikát: Sikeres foglalások normál, mozgássérült és elektromos helyre megfelelő paraméterekkel. Hibakezelés: nem létező parkoló, jogosulatlan mozgássérült hely foglalása, jogosulatlan elektromos hely foglalása, illetve időpont-ütközés. 2. ParkingSpaceServiceTests Szintén In-Memory adatbázissal teszteld a lekérdezéseket: Összes parkolóhely sikeres lekérdezése. Konkrét parkolóhely sikeres lekérdezése ID alapján. Nem létező ID lekérdezésének helyes kezelése. 3. ReservationsControllerTests és ParkingSpacesControllerTests Használd a Moq könyvtárat a szervizek mockolásához. Teszteld le a kontrollerek összes GET és POST végpontját, hogy megfelelő HTTP státusz kódokat adnak-e vissza.

Response: Örömmel elkészítem az átfogó xUnit teszteket a projekthez. A tesztek megírásánál az iparági "Arrange-Act-Assert" (AAA) mintát követjük.

A teszteléshez egy külön tesztprojektet (pl. ParkingReservation.Tests) érdemes létrehozni, amibe telepíteni kell a következő NuGet csomagokat:

- xunit
- xunit.runner.visualstudio
- Microsoft.EntityFrameworkCore.InMemory
- Moq

Megjegyzés: A kérésedben szerepelt a parkolóhely ID alapján történő lekérdezésének tesztelése. Mivel az eddigi kódbázisban a ParkingSpaceService-ben nem volt GetByIdAsync metódus, a tesztekben feltételezem, hogy ezt a metódust (amely egy ParkingSpaceDto?-t ad vissza) hozzáadtad az

IParkingSpaceService interfészhez és a megvalósításhoz.

Természetesen a kódokban **továbbra is szigorúan mellőzöm a var kulcsszó használatát.**

1. ReservationServiceTests.cs

(Mappa: Tests/Services/)

Ez az osztály az In-Memory adatbázist használja, így minden tesztfüggvény egy teljesen szeparált, egyedi azonosítójú adatbázist kap a Guid.NewGuid().ToString() segítségével, elkerülve a tesztek közötti állapot-szivárgást.

```
using System;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using ParkingReservation.Application.DTOs;
using ParkingReservation.Application.Services;
using ParkingReservation.Domain.Entities;
using ParkingReservation.Domain.Enums;
using ParkingReservation.Infrastructure.Data;
using Xunit;

namespace ParkingReservation.Tests.Services;

public class ReservationServiceTests
{
    private AppDbContext GetInMemoryDbContext()
    {
        DbContextOptions<AppDbContext> options = new DbContextOptionsBuilder<AppDbContext>()
            .UseInMemoryDatabase(databaseName: Guid.NewGuid().ToString())
            .Options;

        return new AppDbContext(options);
    }

    [Fact]
    public async Task CreateAsync_ValidStandardSpace_ReturnsDto()
    {
        // Arrange
        AppDbContext context = GetInMemoryDbContext();
        context.ParkingSpaces.Add(new ParkingSpace { Id = 1, Type = ParkingSpaceType.Standard });
        await context.SaveChangesAsync();

        ReservationService service = new ReservationService(context);
        CreateReservationDto dto = new CreateReservationDto
        {
            ParkingSpaceId = 1,
            ApplicantName = "Teszt Elek",
            StartTime = DateTimeOffset.UtcNow.AddDays(1),
            EndTime = DateTimeOffset.UtcNow.AddDays(1).AddHours(2)
        };
    }
}
```

```

// Act
ReservationDto? result = await service.CreateAsync(dto);

// Assert
Assert.NotNull(result);
Assert.Equal("Teszt Elek", result.ApplicantName);
Assert.Equal(1, result.ParkingSpaceId);
}

[Fact]
public async Task CreateAsync_AccessibleSpaceWithPermit_ReturnsDto()
{
    // Arrange
    AppDbContext context = GetInMemoryDbContext();
    context.ParkingSpaces.Add(new ParkingSpace { Id = 1, Type = ParkingSpaceType.Accessible });
    await context.SaveChangesAsync();

    ReservationService service = new ReservationService(context);
    CreateReservationDto dto = new CreateReservationDto
    {
        ParkingSpaceId = 1,
        ApplicantName = "Teszt Elek",
        StartTime = DateTimeOffset.UtcNow.AddDays(1),
        EndTime = DateTimeOffset.UtcNow.AddDays(1).AddHours(2),
        HasSpecialPermit = true
    };

    // Act
    ReservationDto? result = await service.CreateAsync(dto);

    // Assert
    Assert.NotNull(result);
    Assert.Equal(1, result.ParkingSpaceId);
}

[Fact]
public async Task CreateAsync_ElectricSpaceWithVehicle_ReturnsDto()
{
    // Arrange
    AppDbContext context = GetInMemoryDbContext();
    context.ParkingSpaces.Add(new ParkingSpace { Id = 1, Type = ParkingSpaceType.Electric });
    await context.SaveChangesAsync();

    ReservationService service = new ReservationService(context);
    CreateReservationDto dto = new CreateReservationDto
    {
        ParkingSpaceId = 1,
        ApplicantName = "Teszt Elek",
        StartTime = DateTimeOffset.UtcNow.AddDays(1),
        EndTime = DateTimeOffset.UtcNow.AddDays(1).AddHours(2),
        HasElectricVehicle = true
    };

```

```

};

// Act
ReservationDto? result = await service.CreateAsync(dto);

// Assert
Assert.NotNull(result);
}

[Fact]
public async Task CreateAsync_SpaceNotFound_ReturnsNull()
{
    // Arrange
    AppDbContext context = GetInMemoryDbContext();
    ReservationService service = new ReservationService(context);
    CreateReservationDto dto = new CreateReservationDto
    {
        ParkingSpaceId = 999, // Nem létező ID
        ApplicantName = "Teszt Elek",
        StartTime = DateTimeOffset.UtcNow.AddDays(1),
        EndTime = DateTimeOffset.UtcNow.AddDays(1).AddHours(2)
    };

    // Act
    ReservationDto? result = await service.CreateAsync(dto);

    // Assert
    Assert.Null(result);
}

[Fact]
public async Task CreateAsync_AccessibleSpaceWithoutPermit_ThrowsInvalidOperationException()
{
    // Arrange
    AppDbContext context = GetInMemoryDbContext();
    context.ParkingSpaces.Add(new ParkingSpace { Id = 1, Type = ParkingSpaceType.Accessible });
    await context.SaveChangesAsync();

    ReservationService service = new ReservationService(context);
    CreateReservationDto dto = new CreateReservationDto
    {
        ParkingSpaceId = 1,
        ApplicantName = "Teszt Elek",
        StartTime = DateTimeOffset.UtcNow.AddDays(1),
        EndTime = DateTimeOffset.UtcNow.AddDays(1).AddHours(2),
        HasSpecialPermit = false // Hiányzó engedély
    };

    // Act & Assert
    await Assert.ThrowsAsync<InvalidOperationException>(() => service.CreateAsync(dto));
}

```


[Fact]

```
public async Task CreateAsync_ElectricSpaceWithoutVehicle_ThrowsInvalidOperationException()
{
    // Arrange
    AppDbContext context = GetInMemoryDbContext();
    context.ParkingSpaces.Add(new ParkingSpace { Id = 1, Type = ParkingSpaceType.Electric });
    await context.SaveChangesAsync();

    ReservationService service = new ReservationService(context);
    CreateReservationDto dto = new CreateReservationDto
    {
        ParkingSpaceId = 1,
        ApplicantName = "Teszt Elek",
        StartTime = DateTimeOffset.UtcNow.AddDays(1),
        EndTime = DateTimeOffset.UtcNow.AddDays(1).AddHours(2),
        HasElectricVehicle = false // Hiányzó EV jelzés
    };

    // Act & Assert
    await Assert.ThrowsAsync<InvalidOperationException>(() => service.CreateAsync(dto));
}
```

[Fact]

```
public async Task CreateAsync_OverlappingTime_ThrowsInvalidOperationException()
{
    // Arrange
    AppDbContext context = GetInMemoryDbContext();
    context.ParkingSpaces.Add(new ParkingSpace { Id = 1, Type = ParkingSpaceType.Standard });
    context.Reservations.Add(new Reservation
    {
        Id = 1,
        ParkingSpaceId = 1,
        ApplicantName = "Korábbi Foglaló",
        StartTime = DateTimeOffset.UtcNow.AddDays(1).AddHours(10),
        EndTime = DateTimeOffset.UtcNow.AddDays(1).AddHours(12)
    });
    await context.SaveChangesAsync();

    ReservationService service = new ReservationService(context);
    CreateReservationDto dto = new CreateReservationDto
    {
        ParkingSpaceId = 1,
        ApplicantName = "Új Foglaló",
        // Ütközik a korábbival (11:00 - 13:00)
        StartTime = DateTimeOffset.UtcNow.AddDays(1).AddHours(11),
        EndTime = DateTimeOffset.UtcNow.AddDays(1).AddHours(13)
    };

    // Act & Assert
    await Assert.ThrowsAsync<InvalidOperationException>(() => service.CreateAsync(dto));
}
}
```

2. ParkingSpaceServiceTests.cs

(Mappa: Tests/Services/)

```
using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Microsoft.EntityFrameworkCore;
using ParkingReservation.Application.DTOs;
using ParkingReservation.Application.Services;
using ParkingReservation.Domain.Entities;
using ParkingReservation.Domain.Enums;
using ParkingReservation.Infrastructure.Data;
using Xunit;

namespace ParkingReservation.Tests.Services;

public class ParkingSpaceServiceTests
{
    private AppDbContext GetInMemoryDbContext()
    {
        DbContextOptions<AppDbContext> options = new DbContextOptionsBuilder<AppDbContext>()
            .UseInMemoryDatabase(databaseName: Guid.NewGuid().ToString())
            .Options;

        return new AppDbContext(options);
    }

    [Fact]
    public async Task GetAllAsync_ReturnsAllSpaces()
    {
        // Arrange
        AppDbContext context = GetInMemoryDbContext();
        context.ParkingSpaces.AddRange(
            new ParkingSpace { Id = 1, Type = ParkingSpaceType.Standard },
            new ParkingSpace { Id = 2, Type = ParkingSpaceType.Accessible }
        );
        await context.SaveChangesAsync();

        ParkingSpaceService service = new ParkingSpaceService(context);

        // Act
        List<ParkingSpaceDto> result = await service.GetAllAsync();

        // Assert
        Assert.Equal(2, result.Count);
    }
}

// Ezt a metódust csak akkor futtasd, ha a ParkingSpaceService implementálta a GetByIdAsync metódust.
```

```

/*
[Fact]
public async Task GetByIdAsync_ExistingId_ReturnsSpaceDto()
{
    // Arrange
    AppDbContext context = GetInMemoryDbContext();
    context.ParkingSpaces.Add(new ParkingSpace { Id = 1, Type = ParkingSpaceType.Standard });
    await context.SaveChangesAsync();

    ParkingSpaceService service = new ParkingSpaceService(context);

    // Act
    ParkingSpaceDto? result = await service.GetByIdAsync(1);

    // Assert
    Assert.NotNull(result);
    Assert.Equal(1, result.Id);
}

[Fact]
public async Task GetByIdAsync_NonExistingId_ReturnsNull()
{
    // Arrange
    AppDbContext context = GetInMemoryDbContext();
    ParkingSpaceService service = new ParkingSpaceService(context);

    // Act
    ParkingSpaceDto? result = await service.GetByIdAsync(999);

    // Assert
    Assert.Null(result);
}
*/
}

```

3. ReservationsControllerTests.cs

(Mappa: Tests/Controllers/)

Itt a Moq könyvtárat használjuk az üzleti logika elfedésére, hogy kizárólag a kontroller HTTP státuszkódokra adott válaszait teszteljük.

```

using System;
using System.Collections.Generic;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Moq;
using ParkingReservation.Application.DTOs;
using ParkingReservation.Application.Services;
using ParkingReservation.Controllers;

```

```

using Xunit;

namespace ParkingReservation.Tests.Controllers;

public class ReservationsControllerTests
{
    [Fact]
    public async Task GetAll_ReturnsOkResult()
    {
        // Arrange
        Mock<IReservationService> mockService = new Mock<IReservationService>();
        mockService.Setup(s => s.GetAllAsync()).ReturnsAsync(new List<ReservationDto>());

        ReservationsController controller = new ReservationsController(mockService.Object);

        // Act
        ActionResult<List<ReservationDto>> actionResult = await controller.GetAll();

        // Assert
        OkObjectResult? result = actionResult.Result as OkObjectResult;
        Assert.NotNull(result);
        Assert.Equal(200, result.StatusCode);
    }

    [Fact]
    public async Task Create_ValidRequest_ReturnsCreatedAtAction()
    {
        // Arrange
        Mock<IReservationService> mockService = new Mock<IReservationService>();
        CreateReservationDto createDto = new CreateReservationDto
        {
            ParkingSpaceId = 1,
            ApplicantName = "Teszt",
            StartTime = DateTimeOffset.UtcNow,
            EndTime = DateTimeOffset.UtcNow.AddHours(1)
        };
        ReservationDto createdDto = new ReservationDto { Id = 1, ParkingSpaceId = 1, ApplicantName = "Teszt" };

        mockService.Setup(s => s.CreateAsync(createDto)).ReturnsAsync(createdDto);

        ReservationsController controller = new ReservationsController(mockService.Object);

        // Act
        ActionResult<ReservationDto> actionResult = await controller.Create(createDto);

        // Assert
        CreatedAtActionResult? result = actionResult.Result as CreatedAtActionResult;
        Assert.NotNull(result);
        Assert.Equal(201, result.StatusCode);
        Assert.Equal(createdDto, result.Value);
    }
}

```

[Fact]

```
public async Task Create_SpaceNotFound_ReturnsBadRequest()
```

```
{  
    // Arrange  
    Mock<IReservationService> mockService = new Mock<IReservationService>();  
    CreateReservationDto createDto = new CreateReservationDto  
    {  
        ParkingSpaceId = 999,  
        ApplicantName = "Teszt",  
        StartTime = DateTimeOffset.UtcNow,  
        EndTime = DateTimeOffset.UtcNow.AddHours(1)  
    };  
  
    // A szerviz null-t ad vissza, ha nem létezik a parkoló  
    mockService.Setup(s => s.CreateAsync(createDto)).ReturnsAsync((ReservationDto?)null);  
  
    ReservationsController controller = new ReservationsController(mockService.Object);  
  
    // Act  
    ActionResult<ReservationDto> actionResult = await controller.Create(createDto);  
  
    // Assert  
    BadRequestObjectResult? result = actionResult.Result as BadRequestObjectResult;  
    Assert.NotNull(result);  
    Assert.Equal(400, result.StatusCode);  
}
```

[Fact]

```
public async Task Create_Conflict_ReturnsConflict()
```

```
{  
    // Arrange  
    Mock<IReservationService> mockService = new Mock<IReservationService>();  
    CreateReservationDto createDto = new CreateReservationDto  
    {  
        ParkingSpaceId = 1,  
        ApplicantName = "Teszt",  
        StartTime = DateTimeOffset.UtcNow,  
        EndTime = DateTimeOffset.UtcNow.AddHours(1)  
    };  
  
    // A szerviz kivételt dob ütközés esetén  
    mockService.Setup(s => s.CreateAsync(createDto)).ThrowsAsync(new  
InvalidOperationException("Ütközés"));  
  
    ReservationsController controller = new ReservationsController(mockService.Object);  
  
    // Act  
    ActionResult<ReservationDto> actionResult = await controller.Create(createDto);  
  
    // Assert  
    ConflictObjectResult? result = actionResult.Result as ConflictObjectResult;
```

```

        Assert.NotNull(result);
        Assert.Equal(409, result.StatusCode);
    }
}

```

4. ParkingSpacesControllerTests.cs

(Mappa: Tests/Controllers/)

```

using System.Collections.Generic;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc;
using Moq;
using ParkingReservation.Application.DTOs;
using ParkingReservation.Application.Services;
using ParkingReservation.Controllers;
using Xunit;

namespace ParkingReservation.Tests.Controllers;

public class ParkingSpacesControllerTests
{
    [Fact]
    public async Task GetAll_ReturnsOkResult()
    {
        // Arrange
        Mock<IParkingSpaceService> mockParkingService = new Mock<IParkingSpaceService>();
        Mock<IReservationService> mockReservationService = new Mock<IReservationService>();

        mockParkingService.Setup(s => s.GetAllAsync()).ReturnsAsync(new List<ParkingSpaceDto>());

        ParkingSpacesController controller = new ParkingSpacesController(
            mockParkingService.Object,
            mockReservationService.Object);

        // Act
        ActionResult<List<ParkingSpaceDto>> actionResult = await controller.GetAll();

        // Assert
        OkObjectResult? result = actionResult.Result as OkObjectResult;
        Assert.NotNull(result);
        Assert.Equal(200, result.StatusCode);
    }

    [Fact]
    public async Task GetReservationsByParkingSpace_SpaceExists_ReturnsOk()
    {
        // Arrange
        Mock<IParkingSpaceService> mockParkingService = new Mock<IParkingSpaceService>();
        Mock<IReservationService> mockReservationService = new Mock<IReservationService>();
    }
}

```

```

List<ParkingSpaceDto> spaces = new List<ParkingSpaceDto>
{
    new ParkingSpaceDto { Id = 1 }
};
List<ReservationDto> reservations = new List<ReservationDto>
{
    new ReservationDto { Id = 1, ParkingSpaceId = 1, ApplicantName = "Teszt" }
};

mockParkingService.Setup(s => s.GetAllAsync()).ReturnsAsync(spaces);
mockReservationService.Setup(s => s.GetByParkingSpaceIdAsync(1)).ReturnsAsync(reservations);

ParkingSpacesController controller = new ParkingSpacesController(
    mockParkingService.Object,
    mockReservationService.Object);

// Act
ActionResult<List<ReservationDto>> actionResult = await
controller.GetReservationsByParkingSpace(1);

// Assert
OkObjectResult? result = actionResult.Result as OkObjectResult;
Assert.NotNull(result);
Assert.Equal(200, result.StatusCode);

List<ReservationDto>? returnedReservations = result.Value as List<ReservationDto>;
Assert.NotNull(returnedReservations);
Assert.Single(returnedReservations);
}

[Fact]
public async Task GetReservationsByParkingSpace_SpaceNotFound_ReturnsNotFound()
{
    // Arrange
    Mock<IParkingSpaceService> mockParkingService = new Mock<IParkingSpaceService>();
    Mock<IReservationService> mockReservationService = new Mock<IReservationService>();

    // Üres lista, tehát nincs 1-es ID-jú parkoló
    List<ParkingSpaceDto> spaces = new List<ParkingSpaceDto>();

    mockParkingService.Setup(s => s.GetAllAsync()).ReturnsAsync(spaces);

    ParkingSpacesController controller = new ParkingSpacesController(
        mockParkingService.Object,
        mockReservationService.Object);

    // Act
    ActionResult<List<ReservationDto>> actionResult = await
controller.GetReservationsByParkingSpace(1);

    // Assert

```

```

        NotFoundObjectResult? result = actionResult.Result as NotFoundObjectResult;
        Assert.NotNull(result);
        Assert.Equal(404, result.StatusCode);
    }
}

```

User prompt: Kérlek, készíts egy integrációs tesztosztályt ReservationIntegrationTests néven, amely az xUnit és a Microsoft.AspNetCore.Mvc.Testing.WebApplicationFactory segítségével teszteli az API-t. A tesztosztály feladatai: Indítsa el az API-t egy memóriában futó tesztszerveren. A konfiguráció során cserélje le az eredeti AppDbContext-et egy InMemory adatbázisra, hogy a tesztek ne a valós adatbázist módosítsák. Készítsen egy érvényes CreateReservationDto objektumot, serializálja JSON formátumba, majd küldjön egy HTTP POST kérést a /api/reservations végpontra a HttpClient segítségével. Ellenőrizze (Assert), hogy a válasz HTTP státuszkódja sikeres-e (pl. 200 OK vagy 201 Created).

Response: Örömmel elkészítem neked az integrációs tesztet. Az ASP.NET Core alkalmazások integrációs teszteléséhez a Microsoft.AspNetCore.Mvc.Testing csomagra lesz szükséged, amelyet hozzá kell adnod a tesztprojekthez.

Mivel a modern .NET (6+) sablonok "Top-level statements" megközelítést használnak a Program.cs-ben, a Program osztály alapértelmezetten internal láthatóságú, így a tesztprojekt nem éri el. Ennek megoldására a Program.cs fájl legvégére be kell szúrnod ezt az egy sort:

```
public partial class Program { }
```

Miután ez megvan, elkészíthetjük az integrációs tesztet. Ebben a kódban kicseréljük az adatbázis-kontextust egy In-Memory változatra, majd a DI konténerből elkérjük azt, hogy beszurjunk egy teszt parkolóhelyet – enélkül ugyanis a mi üzleti logikánk BadRequest-tel elutasítaná a foglalást.

Természetesen **továbbra is szigorúan figyeltem arra, hogy sehol ne használjam a var kulcsszót.**

ReservationIntegrationTests.cs

(Mappa: Tests/Integration/)

```

using System;
using System.Linq;
using System.Net;
using System.Net.Http;
using System.Text;
using System.Text.Json;
using System.Threading.Tasks;
using Microsoft.AspNetCore.Mvc.Testing;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using ParkingReservation.Application.DTOs;
using ParkingReservation.Domain.Entities;
using ParkingReservation.Domain.Enums;

```



```

using ParkingReservation.Infrastructure.Data;
using Xunit;

namespace ParkingReservation.Tests.Integration;

public class ReservationIntegrationTests : IClassFixture<WebApplicationFactory<Program>>
{
    private readonly WebApplicationFactory<Program> _factory;

    public ReservationIntegrationTests(WebApplicationFactory<Program> factory)
    {
        _factory = factory;
    }

    [Fact]
    public async Task CreateReservation_ValidRequest_ReturnsCreatedStatusCode()
    {
        // --- Arrange ---

        // 1. A WebApplicationFactory konfigurációjának felülbírlása az InMemory adatbázishoz
        WebApplicationFactory<Program> customFactory = _factory.WithWebHostBuilder(builder =>
        {
            builder.ConfigureServices(services =>
            {
                // Megkeressük az eredeti AppDbContext regisztrációját
                ServiceDescriptor? descriptor = services.SingleOrDefault(
                    d => d.ServiceType == typeof(DbContextOptions<AppDbContext>));

                if (descriptor != null)
                {
                    services.Remove(descriptor);
                }

                // Hozzáadjuk az InMemory adatbázist a teszteléshez
                services.AddDbContext<AppDbContext>(options =>
                {
                    options.UseInMemoryDatabase("IntegrationTestDb");
                });
            });
        });

        // 2. HTTP kliens létrehozása a tesztszerverhez
        HttpClient client = customFactory.CreateClient();

        // 3. Adatbázis előkészítése (Seeding): Szükségünk van egy parkolóhelyre, amire foglalhatunk
        using (IServiceScope scope = customFactory.Services.CreateScope())
        {
            IServiceProvider scopedServices = scope.ServiceProvider;
            AppDbContext dbContext = scopedServices.GetRequiredService<AppDbContext>();

            // Biztosítjuk, hogy az adatbázis tiszta lappal induljon, de a sémája meglegyen
            dbContext.Database.EnsureCreated();
        }
    }
}

```

```

bool spaceExists = dbContext.ParkingSpaces.Any(p => p.Id == 1);
if (!spaceExists)
{
    ParkingSpace testSpace = new ParkingSpace
    {
        Id = 1,
        Type = ParkingSpaceType.Standard
    };

    dbContext.ParkingSpaces.Add(testSpace);
    dbContext.SaveChanges();
}
}

```

// 4. Érvényes DTO létrehozása és JSON-né alakítása

```

CreateReservationDto newReservation = new CreateReservationDto
{
    ParkingSpaceId = 1,
    ApplicantName = "Integrációs Tesztelő",
    StartTime = DateTimeOffset.UtcNow.AddDays(5),
    EndTime = DateTimeOffset.UtcNow.AddDays(5).AddHours(3),
    HasSpecialPermit = false,
    HasElectricVehicle = false
};

```

```

string jsonPayload = JsonSerializer.Serialize(newReservation);
StringContent httpContent = new StringContent(jsonPayload, Encoding.UTF8, "application/json");

```

// --- Act ---

```

HttpResponseMessage response = await client.PostAsync("/api/reservations", httpContent);

```

// --- Assert ---

```

// Ellenőrizzük, hogy a kérés sikeres volt-e (200-299 közötti kód)
response.EnsureSuccessStatusCode();

```

```

// Pontosabban is ellenőrizhetjük, hogy a REST elveknek megfelelően 201 Created státuszt kaptunk-e
Assert.Equal(HttpStatusCode.Created, response.StatusCode);

```

```

}
}

```

Amire még szükség lesz

Győződj meg róla, hogy a tesztprojektben (pl. ParkingReservation.Tests.csproj) telepítve van a megfelelő csomag. A terminálban, a tesztprojekt könyvtárából futtasd az alábbi parancsot:

```
dotnet add package Microsoft.AspNetCore.Mvc.Testing
```

Illetve ellenőrizd, hogy a tesztprojekt hivatkozik-e (Project Reference) a fő ParkingReservation projektre. Ezzel teljessé vált a tesztlefedettséged, magába foglalva az egységteszteket és a végponttól-végpontig (End-to-End/Integrációs) tesztet is!
